

Quantifier Elimination Meets Treewidth

Hao Wu*¹ Jiyu Zhu*¹ Amir Goharshady²
Jie An¹ Bican Xia³ Naijun Zhan^{1,3}

¹Institute of Software, Chinese Academy of Sciences

²University of Oxford

³Peking University

TACAS 2026, Turin, Italy

Motivation

- **Quantifier Elimination** plays an important role in verification, synthesis, and symbolic reasoning.
- Reasoning about linear and nonlinear real arithmetic (LRA/NRA):
 - **Fourier–Motzkin Elimination (FME)** for LRA.
 - **Cylindrical Algebraic Decomposition (CAD)** for NRA.
- Powerful but **very expensive**.
 - Worst-case complexity of FME and CAD: **doubly exponential**.
 - The bound has been shown to be **tight**.

Motivation

- **Quantifier Elimination** plays an important role in verification, synthesis, and symbolic reasoning.
- Reasoning about linear and nonlinear real arithmetic (LRA/NRA):
 - **Fourier–Motzkin Elimination (FME)** for LRA.
 - **Cylindrical Algebraic Decomposition (CAD)** for NRA.
- Powerful but **very expensive**.
 - Worst-case complexity of FME and CAD: **doubly exponential**.
 - The bound has been shown to be **tight**.

Motivation

- **Quantifier Elimination** plays an important role in verification, synthesis, and symbolic reasoning.
- Reasoning about linear and nonlinear real arithmetic (LRA/NRA):
 - **Fourier–Motzkin Elimination (FME)** for LRA.
 - **Cylindrical Algebraic Decomposition (CAD)** for NRA.
- Powerful but **very expensive**.
 - Worst-case complexity of FME and CAD: **doubly exponential**.
 - The bound has been shown to be **tight**.

Motivation

- **Quantifier Elimination** plays an important role in verification, synthesis, and symbolic reasoning.
- Reasoning about linear and nonlinear real arithmetic (LRA/NRA):
 - **Fourier–Motzkin Elimination (FME)** for LRA.
 - **Cylindrical Algebraic Decomposition (CAD)** for NRA.
- Powerful but **very expensive**.
 - Worst-case complexity of FME and CAD: **doubly exponential**.
 - The bound has been shown to be **tight**.

Research Question

Question

*Can we design QE algorithms for **certain classes of inputs** with better worst-case complexity upper bounds?*

- Formulas whose variable dependency graphs are **sparse**.
- This naturally inspires us to use techniques from **Parameterized Algorithms**, especially **treewidth**.

Research Question

Question

*Can we design QE algorithms for **certain classes of inputs** with better worst-case complexity upper bounds?*

- Formulas whose variable dependency graphs are **sparse**.
- This naturally inspires us to use techniques from **Parameterized Algorithms**, especially **treewidth**.

Research Question

Question

*Can we design QE algorithms for **certain classes of inputs** with better worst-case complexity upper bounds?*

- Formulas whose variable dependency graphs are **sparse**.
- This naturally inspires us to use techniques from **Parameterized Algorithms**, especially **treewidth**.

Roadmap

- 1 Recall FME algorithm
- 2 Tree Decomposition and Treewidth
- 3 A Dynamic Programming Framework
- 4 Experimental Results
- 5 Conclusion

Input Formula

Quantifier Elimination

Given a formula

$$\Phi \hat{=} \exists x_m, \dots, \exists x_1. \bigwedge_i \varphi_i(x_1, \dots, x_n), \quad m \leq n,$$

to output a quantifier-free formula equivalent to Φ .

- Focus of this work: eliminate **one existential block** efficiently.

Fourier Motzkin Elimination

- Given a conjunction of LRA constraints

$$x_1 - 4x_2 \leq 0 \quad x_1 - x_2 + 2x_3 \geq -5 \quad x_1 + 2x_2 + 3x_3 \leq 20$$

- To eliminate x_1 , **rewrite** constraints into:

$$x_1 \leq \underbrace{4x_2}_{U_1} \quad \underbrace{-5 + x_2 - 2x_3}_{L_1} \leq x_1 \quad x_1 \leq \underbrace{20 - 2x_2 - 3x_3}_{U_2}$$

- Assert that each **lower** bound $\leq$ each **upper** bound

$$-5 + x_2 - 2x_3 \leq 4x_2 \quad -5 + x_2 - 2x_3 \leq 20 - 2x_2 - 3x_3$$

Fourier Motzkin Elimination

- Given a conjunction of LRA constraints

$$x_1 - 4x_2 \leq 0 \quad x_1 - x_2 + 2x_3 \geq -5 \quad x_1 + 2x_2 + 3x_3 \leq 20$$

- To eliminate x_1 , **rewrite** constraints into:

$$x_1 \leq \underbrace{4x_2}_{U_1} \quad \underbrace{-5 + x_2 - 2x_3}_{L_1} \leq x_1 \quad x_1 \leq \underbrace{20 - 2x_2 - 3x_3}_{U_2}$$

- Assert that each **lower** bound $\leq$ each **upper** bound

$$-5 + x_2 - 2x_3 \leq 4x_2 \quad -5 + x_2 - 2x_3 \leq 20 - 2x_2 - 3x_3$$

Fourier Motzkin Elimination

- Given a conjunction of LRA constraints

$$x_1 - 4x_2 \leq 0 \quad x_1 - x_2 + 2x_3 \geq -5 \quad x_1 + 2x_2 + 3x_3 \leq 20$$

- To eliminate x_1 , **rewrite** constraints into:

$$x_1 \leq \underbrace{4x_2}_{U_1} \quad \underbrace{-5 + x_2 - 2x_3}_{L_1} \leq x_1 \quad x_1 \leq \underbrace{20 - 2x_2 - 3x_3}_{U_2}$$

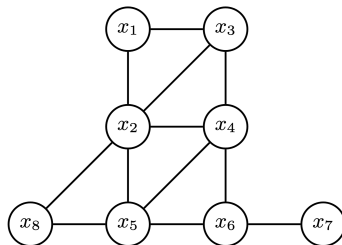
- Assert that each **lower** bound $\leq$ each **upper** bound

$$-5 + x_2 - 2x_3 \leq 4x_2 \quad -5 + x_2 - 2x_3 \leq 20 - 2x_2 - 3x_3$$

Primal Graph (Variable Dependency Graph)

$$\Phi \hat{=} \exists x_1, \dots, x_8.$$

$$\left(\begin{array}{l} x_1 + 2x_2 + 3x_3 \leq 20 \\ \wedge x_1 - x_2 + 2x_3 \geq -5 \\ \wedge x_1 - 4x_2 \leq 0 \\ \wedge \dots (\text{constraints without } x_1) \end{array} \right)$$



- Vertices are **quantified variables**.
- x_i and x_j are linked iff they both occur in an atomic formula.

Parameterized Algorithms

- “Standard” complexity: $\mathcal{O}(f(n))$, where n is the input size.
- **Parameterized complexity** gives a more fine-grained analysis:

$$\mathcal{O}(f(n, k))$$

where the **parameter** k is a secondary measurement of the input (such as **treewidth**).

- For example, some NP-hard problems are **fixed-parameter tractable**:

$$\mathcal{O}(f(k) \cdot n^c)$$

useful when k is small in practice, even if the general problem is hard.

Parameterized Algorithms

- “Standard” complexity: $\mathcal{O}(f(n))$, where n is the input size.
- **Parameterized complexity** gives a more fine-grained analysis:

$$\mathcal{O}(f(n, k))$$

where the **parameter** k is a secondary measurement of the input (such as **treewidth**).

- For example, some NP-hard problems are **fixed-parameter tractable**:

$$\mathcal{O}(f(k) \cdot n^c)$$

useful when k is small in practice, even if the general problem is hard.

Parameterized Algorithms

- “Standard” complexity: $\mathcal{O}(f(n))$, where n is the input size.
- **Parameterized complexity** gives a more fine-grained analysis:

$$\mathcal{O}(f(n, k))$$

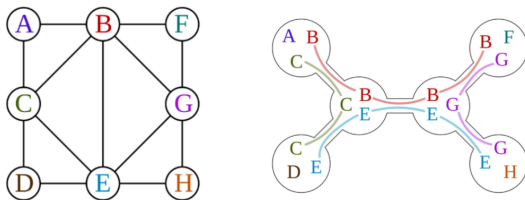
where the **parameter** k is a secondary measurement of the input (such as **treewidth**).

- For example, some NP-hard problems are **fixed-parameter tractable**:

$$\mathcal{O}(f(k) \cdot n^c)$$

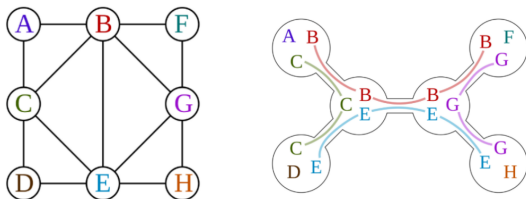
useful when k is small in practice, even if the general problem is hard.

Tree Decomposition



- A **Tree Decomposition (TD)** of G is a tree T :
 - 1 Each node of T , also called a bag, represents a subset of the vertex set of G ;
 - 2 The bags cover the entire vertex set of G ;
 - 3 For every edge in G , there exists a bag b such that both endpoints of the edge are in this bag; and
 - 4 Every vertex in G appears in a connected subtree of T .

Treewidth



- The **width** of a tree decomposition T :

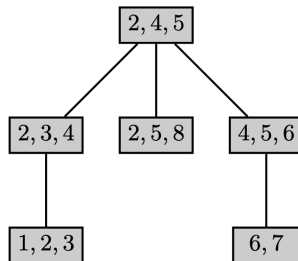
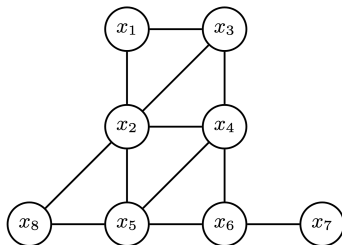
$$w(T) = \max_{b \in \mathcal{B}} |V_b| - 1 \quad (\text{size of the largest bag minus 1})$$

- The **treewidth** of the graph G :

$$tw(G) = \min_T w(T) \quad (\text{minimum width among all TD})$$

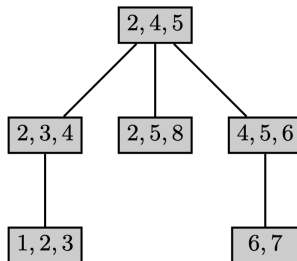
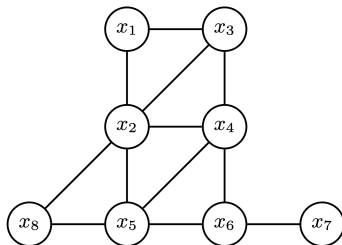
- Intuitively, treewidth quantifies how “tree-like” a graph is.

Key Observations



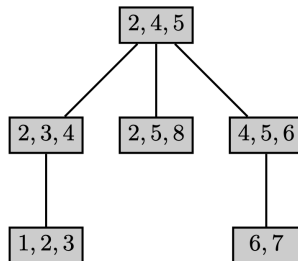
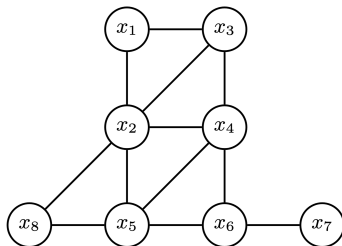
- Eliminating a variable = removing a vertex in the primal graph.
- For vertices that **only occur in leaf bags**, their removal are unrelated to each other (say, x_1, x_8, x_7).

Key Observations



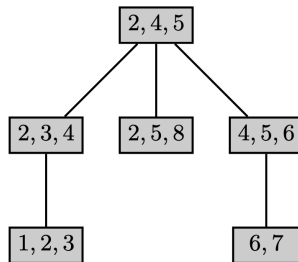
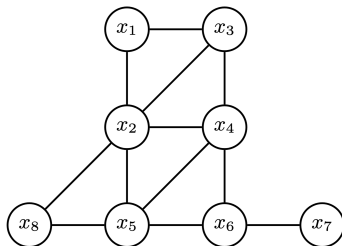
- Eliminating a variable = removing a vertex in the primal graph.
- For vertices that **only occur in leaf bags**, their removal are unrelated to each other (say, x_1, x_8, x_7).

Dynamic Programming



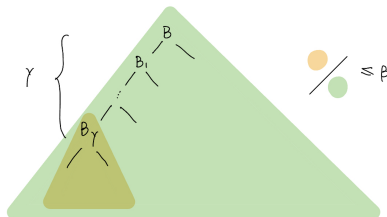
- We design a **dynamic programming** algorithm:
 - 1 To compute a tree decomposition of the primal graph;
 - 2 To eliminate variables in a bottom-up manner according to the tree decomposition structure. ($1, 7, 8 \rightarrow 3, 6 \rightarrow 2, 4, 5$)
- The complexity is determined by the **height** of the TD.

Dynamic Programming



- We design a **dynamic programming** algorithm:
 - 1 To compute a tree decomposition of the primal graph;
 - 2 To eliminate variables in a bottom-up manner according to the tree decomposition structure. ($1, 7, 8 \rightarrow 3, 6 \rightarrow 2, 4, 5$)
- The complexity is determined by the **height** of the TD.

Balanced Tree Decompositions Chatterjee et al'18TOPLAS



Definition (Balanced Tree Decomposition)

For constants $0 < \beta < 1$ and $\gamma \in \mathbb{N}_{>0}$, a binary tree-decomposition T is called **(β, γ) -balanced** if, for each bag b and every descendant b' of b , $\text{Lv}(b') - \text{Lv}(b) = \gamma$ satisfies that $|T_{b'}| \leq \beta \cdot |T_b|$.

- the size of subtree T_b decreases proportionally as $\text{Lv}(b)$ increases.
- A balanced TD of height $\mathcal{O}(\log n)$ can be constructed in $\mathcal{O}(n \log n)$ time.

Improved Complexity Upper Bound

- When the treewidth of the primal graph is a constant, we combine standard analysis techniques of FME/CAD + balanced TDs:
- FME: doubly exponential $\rightarrow$ **single** exponential in the number of **quantified** variables.
- CAD: doubly exponential $\rightarrow$ **single** exponential in the number of **all** variables.
- $\approx$ eliminating $\log(n)$ variables instead of n variables.
- Refer to our paper for a rigorous analysis.

Improved Complexity Upper Bound

- When the treewidth of the primal graph is a constant, we combine standard analysis techniques of FME/CAD + balanced TDs:
- FME: doubly exponential $\rightarrow$ **single** exponential in the number of **quantified** variables.
- CAD: doubly exponential $\rightarrow$ **single** exponential in the number of **all** variables.
- $\approx$ eliminating $\log(n)$ variables instead of n variables.
- Refer to our paper for a rigorous analysis.

Improved Complexity Upper Bound

- When the treewidth of the primal graph is a constant, we combine standard analysis techniques of FME/CAD + balanced TDs:
- FME: doubly exponential $\rightarrow$ **single** exponential in the number of **quantified** variables.
- CAD: doubly exponential $\rightarrow$ **single** exponential in the number of **all** variables.
- $\approx$ eliminating $\log(n)$ variables instead of n variables.
- Refer to our paper for a rigorous analysis.

Experimental Setup

- **Implementation:** design a heuristic method to construct TD with lower height.
 - Balanced TD construction **does not perform well** on small instances.
- **Benchmarks:** randomly generated LRA/NRA instances with controlled treewidth.
- **Comparison:** heuristics for suggesting variable elimination ordering in FME/CAD.

Experimental Setup

- **Implementation:** design a heuristic method to construct TD with lower height.
 - Balanced TD construction **does not perform well** on small instances.
- **Benchmarks:** randomly generated LRA/NRA instances with controlled treewidth.
- **Comparison:** heuristics for suggesting variable elimination ordering in FME/CAD.

Experimental Setup

- **Implementation:** design a heuristic method to construct TD with lower height.
 - Balanced TD construction **does not perform well** on small instances.
- **Benchmarks:** randomly generated LRA/NRA instances with controlled treewidth.
- **Comparison:** heuristics for suggesting variable elimination ordering in FME/CAD.

FME Experimental Results

Table 1: FME performance on randomly generated LRA instances

ID	Instance Set (10 instances per set)	Random	Greedy	Ours
1	#var=15 #ineq=75 Ave. ineq. count	51,608.4	4,404.7	4,841.3
	#elim=5 #tw=2 Ave. run Time(s)	0.21	0.018	0.018
2	#var=15 #ineq=75 Ave. ineq. count	>10,000,000	28,362.3	28,244.8
	#elim=5 #tw=4 Ave. run Time(s)	NA	0.17	0.17
3	#var=20 #ineq=100 Ave. ineq. count	>10,000,000	290,597.8	194,645.3
	#elim=10 #tw=2 Ave. run Time(s)	NA	2.0	1.1
4	#var=20 #ineq=100 Ave. ineq. count	>10,000,000	313,584.0	313,585.9
	#elim=6 #tw=4 Ave. run Time(s)	NA	2.6	2.4
5	#var=30 #ineq=150 Ave. ineq. count	>10,000,000	343,705.4	110,304.9
	#elim=12 #tw=3 Ave. run Time(s)	NA	3.7	1.2
6	#var=30 #ineq=150 Ave. ineq. count	>10,000,000	974,233.4	974,233.4
	#elim=5 #tw=8 Ave. run Time(s)	NA	14.9	14.4

- Comparable with the Greedy heuristic.

CAD Experimental Results

Table 2: CAD performance on randomly generated NRA instances

ID	Instance	SVO	Brown	PEO	Ours
7	#var=6 #ineq=6 Cells	1,133,532	271,120	274,724	110,828
	#max_deg=2 #tw=2 Time(s)	640.3	307.0	114.7	40.5
8	#var=7 #ineq=4 Cells	126,328	89,804	89,804	49,108
	#max_deg=3 #tw=3 Time(s)	42.1	34.5	29.0	11.6
9	#var=7 #ineq=5 Cells	2,020,378	1,926,208	740,380	181,090
	#max_deg=2 #tw=2 Time(s)	702.0	416.2	96.1	84.1
10	#var=8 #ineq=5 Cells	3,721,372	3,721,372	984,008	2,155,132
	#max_deg=4 #tw=2 Time(s)	1843.8	1718.2	567.7	1090.3
11	#var=8 #ineq=6 Cells	38,212	69,748	69,748	38,212
	#max_deg=2 #tw=3 Time(s)	42.0	633.3	427.9	42.0
12	#var=9 #ineq=10 Cells	1,804,224	1,804,224	954,432	52,992
	#max_deg=2 #tw=3 Time(s)	742.3	746.7	156.7	83.3

- Significantly better than other heuristics, in terms of both **time** and **space** (number of cells to be stored).

Conclusion

- **Take home message:** **treewidth sparsity** can be exploited to design more efficient QE algorithms.
- **Limitation:** **Not practical**. Sparsity is often present in large scale problems, but QE is too expensive for them.
- **Future direction:** Exploit sparsity in **constraint solving and SMT** procedures, and find **practical applications**.

Thank you! Questions?

Conclusion

- **Take home message:** **treewidth sparsity** can be exploited to design more efficient QE algorithms.
- **Limitation:** **Not practical.** Sparsity is often present in large scale problems, but QE is too expensive for them.
- **Future direction:** Exploit sparsity in **constraint solving and SMT** procedures, and find **practical applications**.

Thank you! Questions?

Conclusion

- **Take home message:** **treewidth sparsity** can be exploited to design more efficient QE algorithms.
- **Limitation:** **Not practical**. Sparsity is often present in large scale problems, but QE is too expensive for them.
- **Future direction:** Exploit sparsity in **constraint solving and SMT** procedures, and find **practical applications**.

Thank you! Questions?

Conclusion

- **Take home message:** **treewidth sparsity** can be exploited to design more efficient QE algorithms.
- **Limitation:** **Not practical**. Sparsity is often present in large scale problems, but QE is too expensive for them.
- **Future direction:** Exploit sparsity in **constraint solving and SMT** procedures, and find **practical applications**.

Thank you! Questions?